

SENA

TECNÓLOGO ANÁLISIS Y DESARROLLO DE SOFTWARE

MATERIA:

Actividad de proyecto 4

Evidencia:

GA4-220501095-AA2-EV06. Taller. Arquitectura de software.

AUTOR:

Gerson Jonnathan López Oviedo – 3186647

DOCENTE:

Arnaldo Alfonso Montiel Brun

BOGOTÁ D.C.

20 de noviembre del 2025

INDICE:

ARQUITECTURA DE SOFTWARE

1 INTRODUCCIÓN

2 DEFINICIÓN

3. ¿CUÁL ES SU FUNCIÓN?

4. ¿CÓMO SE ELABORA LA ARQUITECTURA DE SOFTWARE?

5. ¿CÓMO LOGRAR UNA BUENA ARQUITECTURA?

PRINCIPIOS FUNDAMENTALES

6. CHECKLIST: ¿TIENES UNA BUENA ARQUITECTURA?

7. ELEMENTOS DE DISEÑO DE UNA ARQUITECTURA DE SOFTWARE

8. TIPOS DE ARQUITECTURA DE SOFTWARE

**9. ESCALABILIDAD EN ARQUITECTURA DE SOFTWARE (EXPLICACIÓN
GENERAL Y RESUMIDA)**

INTRODUCCIÓN

La arquitectura de software es uno de los elementos fundamentales en el desarrollo de aplicaciones modernas, ya que define la estructura, organización y funcionamiento interno de un sistema. Comprender este concepto permite planificar, diseñar y construir soluciones tecnológicas más eficientes, seguras y fáciles de mantener. Aprender sobre arquitectura de software es importante porque garantiza que los proyectos crezcan de manera ordenada, soporten más usuarios, faciliten la incorporación de nuevas funciones y eviten problemas futuros relacionados con el rendimiento o la calidad del software.

Esta actividad tiene como propósito analizar de forma clara los principales conceptos relacionados con la arquitectura de software, comprender sus funciones, los procesos para elaborarla, los elementos que la componen y la importancia de la escalabilidad dentro del sistema. Dominar estos conocimientos permite al aprendiz tomar mejores decisiones técnicas, diseñar soluciones bien estructuradas y aplicar buenas prácticas que son esenciales en la industria del desarrollo de software.

ARQUITECTURA DE SOFTWARE

2 DEFINICIÓN

Definición Simple:

La arquitectura de software es como el plano de una casa, pero para programas de computador. Así como un arquitecto diseña cómo será una casa (dónde van las habitaciones, los baños, la cocina), un arquitecto de software diseña cómo será un sistema informático.

Definición Técnica:

La arquitectura de software es el diseño de alto nivel que muestra la estructura fundamental y general de un sistema antes de escribir el código y comprende:

- Los componentes del software (las piezas)
- Las relaciones entre esos componentes (cómo se conectan)
- Los principios que guían su diseño y evolución

Analogía:

Imagina que vas a construir un edificio:

Construcción de Edificio	Arquitectura de Software
Planos arquitectónicos	Diagramas UML
Cimientos	Base de datos
Columnas y vigas	Clases y objetos principales
Habitaciones	Módulos del sistema
Puertas y ventanas	Interfaces (API)
Instalación eléctrica	Flujo de datos
Instalación de agua	Comunicación entre componentes

3. ¿CUÁL ES SU FUNCIÓN?

A) Servir como Guía de Construcción

- Proporciona un mapa para que todos los desarrolladores sepan qué construir y cómo
- Define la ruta a seguir durante todo el proyecto

Ejemplo: Si 5 programadores trabajan en el mismo proyecto, la arquitectura evita que cada uno haga las cosas de manera diferente.

B) Facilitar la Comunicación

- Permite que clientes, desarrolladores y stakeholders hablen el mismo idioma
- Muestra visualmente cómo funcionará el sistema

Ejemplo: Puedes mostrarle al cliente un diagrama en lugar de miles de líneas de código.

C) Tomar Decisiones Técnicas Importantes

- Define qué tecnologías usar (bases de datos, lenguajes, frameworks)
- Establece cómo se organizará el código
- Determina cómo se comunicarán las partes del sistema

Ejemplo: Decidir si usar MySQL o PostgreSQL, si programar en Java o Python.

D) Garantizar Calidad

Asegura que el sistema cumpla con requisitos importantes:

Requisito	Qué significa	Ejemplo práctico
Escalabilidad	Que el sistema pueda crecer sin fallar	De 100 usuarios hoy a 10,000 mañana sin colapsar
Mantenibilidad	Que sea fácil de corregir y mejorar	Arreglar errores sin romper otras partes del sistema
Rendimiento	Que responda rápido	Cargar en segundos, no en minutos
Seguridad	Que proteja los datos y el acceso	Nadie puede robar información ni acceder sin permiso
Disponibilidad	Que esté siempre funcionando	Accesible 24/7 sin caídas prolongadas

E) Reducir Riesgos

- Identifica problemas potenciales antes de que ocurran
- Permite hacer cambios sin afectar todo el sistema

Ejemplo: Si necesitas cambiar la base de datos, una buena arquitectura permite hacerlo sin reescribir todo el código.

4. ¿CÓMO SE ELABORA LA ARQUITECTURA DE SOFTWARE?

Proceso Paso a Paso:

- **PASO 1:** Entender el Problema

¿Qué hacer?

- Reunirse con el cliente y usuarios
- Levantar requisitos (qué necesita el sistema)
- Identificar restricciones (limitaciones de tiempo, dinero, tecnología)

Herramientas:

- Entrevistas
- Casos de uso
- Historias de usuario

Ejemplo:

Cliente: "Necesito un sistema de facturación para mi ferretería"

Requisitos identificados:

- ✓ Registrar productos
- ✓ Hacer facturas
- ✓ Controlar inventario
- ✓ Generar reportes

✓ Máximo 50 usuarios simultáneos

- **PASO 2: Identificar los Componentes Principales**

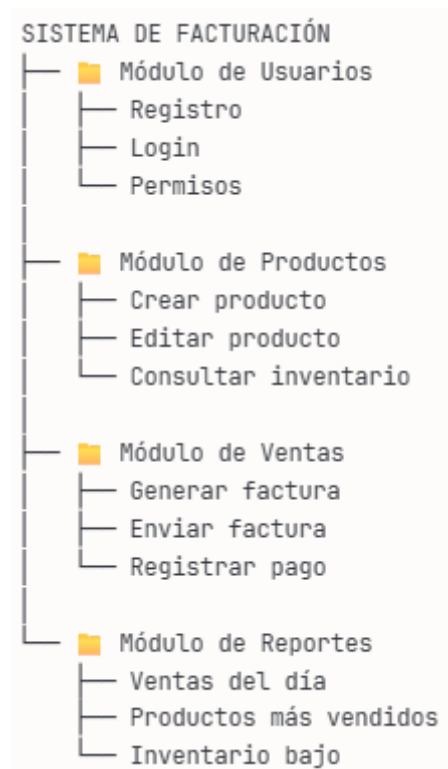
¿Qué hacer?

Dividir el sistema en partes lógicas (módulos)

Aplicando el Principio de Modularidad (del documento CF10):

- Cada módulo debe tener una responsabilidad clara
- Los módulos deben ser independientes (bajo acoplamiento)
- Cada módulo debe ser altamente funcional (alta cohesión)

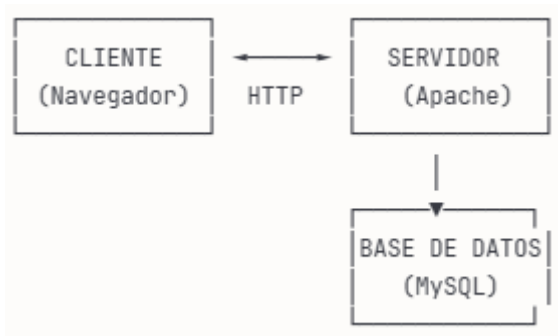
Ejemplo del Sistema de Facturación:



- **PASO 3: Definir la Arquitectura**

Elegir un Patrón Arquitectónico:

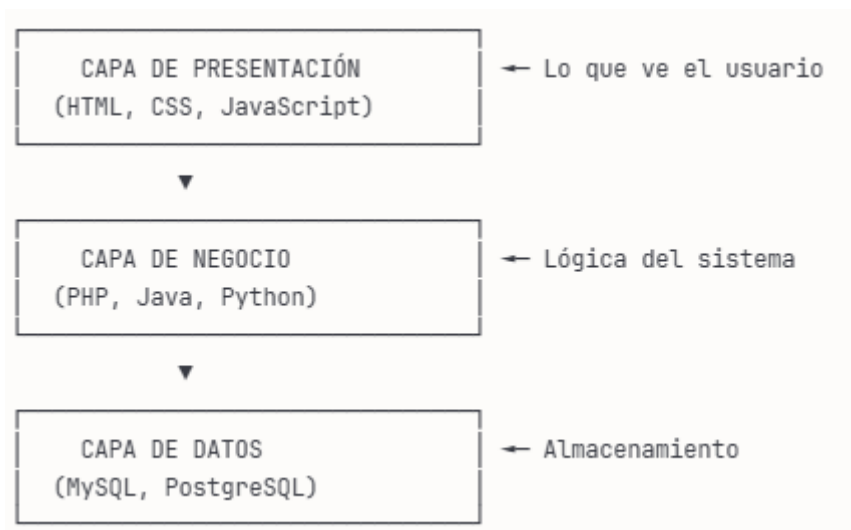
A) Arquitectura Cliente-Servidor



Características:

- **Cliente hace peticiones**
- **Servidor procesa y responde**
- **Base de datos almacena información**

B) Arquitectura en Capas



Ventajas:

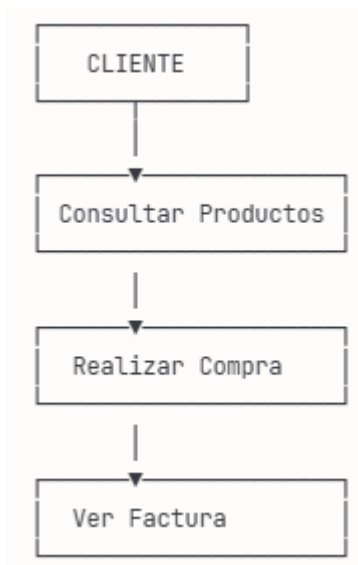
- Separación clara de responsabilidades
- Fácil de mantener
- Puedes cambiar una capa sin afectar las otras

- **PASO 4: Crear los Diagramas**

Según los documentos del SENA, necesitas:

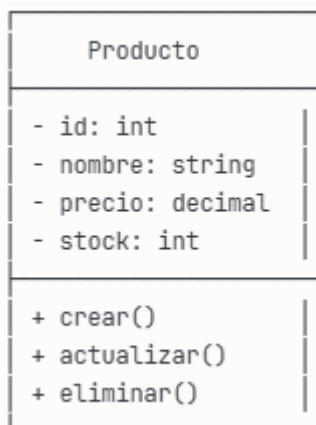
a) Diagrama de Casos de Uso (CF04)

muestra qué hace el sistema y quién lo usa



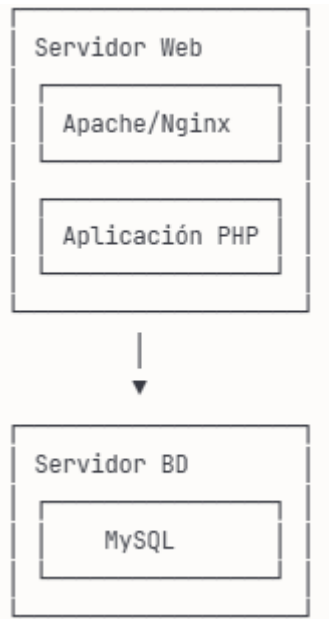
b) Diagrama de Clases

Muestra **CÓMO** se estructuran los datos



c) Diagrama de Componentes (CF10)

Muestra los módulos del sistema y sus relaciones



- **PASO 5: Aplicar Patrones de Diseño**

Patrones Creacionales: Cómo crear objetos

- **Singleton:** Una sola instancia de conexión a BD

Patrones Estructurales: Cómo organizar el código

- **Fachada:** Simplificar interfaces complejas

Patrones Comportamentales: Cómo interactúan los objetos

- **Estrategia:** Diferentes formas de calcular (descuentos, impuestos)

- **PASO 6: Documentar**

Crear documentación que incluya:

- ✓ Descripción general del sistema
- ✓ Diagramas (casos de uso, clases, componentes, despliegue)
- ✓ Decisiones técnicas tomadas
- ✓ Tecnologías seleccionadas
- ✓ Restricciones y limitaciones
- ✓ Plan de evolución futura

Ejemplo de Tabla de Decisiones Técnicas:

Aspecto	Tecnología elegida	Justificación
Base de datos	MySQL	Gratuita, robusta, buen soporte
Backend	PHP	El equipo tiene experiencia
Frontend	HTML5 / CSS3 / JS	Estándar web, compatible con todos los navegadores
Servidor	Apache	Fácil de configurar y ampliamente documentado
Patrón arquitectónico	MVC	Separa la lógica de negocio de la presentación

5. ¿CÓMO LOGRAR UNA BUENA ARQUITECTURA?

PRINCIPIOS FUNDAMENTALES

A) MANTÉN LAS COSAS SIMPLES Y ORGANIZADAS

1. Una cosa a la vez (Responsabilidad Única)

- Cada parte del sistema debe hacer UNA SOLA TAREA bien hecha.

Analogía:

- En una tienda, el cajero solo cobra
- El bodeguero solo organiza inventario
- El vendedor solo atiende clientes

Si el cajero intentara hacer todo, sería un caos.

En tu sistema:

- Un módulo de usuarios → solo maneja usuarios
- Un módulo de productos → solo maneja productos
- Un módulo de ventas → solo maneja ventas

2. Separa las responsabilidades (Modularidad)

- **Divide tu sistema en partes independientes que puedan trabajar solas.**

Analogía: Un carro tiene partes separadas:

- Motor
- Sistema eléctrico
- Sistema de frenos
- Sistema de aire acondicionado

Si se daña el aire acondicionado, el carro sigue funcionando. No necesitas cambiar todo el carro, solo esa parte.

En tu sistema: Si necesitas cambiar cómo se envían los correos, solo modificas el módulo de correos, sin tocar el resto del sistema.

B) PIENSA EN EL FUTURO (ESCALABILIDAD)

- **Diseña para crecer**

Tu sistema hoy tiene 10 usuarios, pero mañana puede tener 10,000.

Analogía:

- Una casa pequeña para 4 personas
- VS una casa con buenos cimientos que puedes ampliar después

Preguntas que debes hacerte:

- ✓ ¿Si necesito más usuarios, puedo agregar más servidores?
- ✓ ¿Si crece la base de datos, puedo moverla a otro servidor?
- ✓ ¿Puedo agregar nuevas funcionalidades sin reescribir todo?

C) HAZLO FÁCIL DE MANTENER (MANTENIBILIDAD)

- **Código claro y organizado**

Analogía:

- Una biblioteca bien organizada: encuentras cualquier libro en segundos
- VS una biblioteca desordenada: tardas horas buscando

Cómo lograrlo:**1. Nombres claros**

- MAL: funcion1(), tabla x
- BIEN: registrar Venta (), tabla productos

2. Documentación

- Explica QUÉ hace cada parte
- Explica POR QUÉ tomaste ciertas decisiones

3. Organización lógica

- Agrupa cosas relacionadas
- Separa cosas diferentes

D) OCULTA LA COMPLEJIDAD (ENCAPSULAMIENTO)**Analogía:** Cuando usas tu celular:

- Solo ves la pantalla (interfaz simple)
- No ves los circuitos internos (complejidad oculta)

En tu sistema: El usuario no necesita saber:

- Cómo se conecta a la base de datos
- Qué algoritmo usas para calcular
- Cómo se encripta la contraseña

Solo necesita:

- Un botón de "Ingresar"
- Un formulario claro

- Mensajes comprensibles

E) APLICA JERARQUÍAS (ORGANIZACIÓN)

- **De lo general a lo específico**

Analogía:

ROPA

|— Ropa de Hombre

| |— Camisas

| |— Pantalones

| |— Zapatos

|— Ropa de Mujer

|— Blusas

|— Faldas

|— Tacones

En tu sistema:

USUARIOS

|— Cliente

| |— Cliente Regular

| |— Cliente VIP

|— Empleado

|— Vendedor

|— Administrador

Cada nivel hereda características del anterior y agrega las suyas propias.

F) NO REPITAS INFORMACIÓN (BASE DE DATOS NORMALIZADA)

- **Problema:** Datos duplicados causan errores

Analogía: Imagina que guardas el número de teléfono de tu mamá en:

- Tu agenda del celular
- Un papel en la nevera
- Tu libreta
- Tu computador

Si ella cambia de número, ¡tienes que actualizar en 4 lugares! Y seguro olvidas uno.

Solución: Guarda cada dato UNA SOLA VEZ en un lugar central.

En tu sistema:

- MAL: Guardar el nombre del cliente en cada factura
- BIEN: Guardar el nombre del cliente una vez, y referenciar su ID en las facturas

G) PLANEA PARA LOS ERRORES (MANEJO DE EXCEPCIONES)

- **Las cosas pueden salir mal**

Analogía: Un edificio tiene:

- Extintores para incendios
- Salidas de emergencia
- Alarmas
- Plan de evacuación

No esperas que haya incendio, pero estás preparado.

En tu sistema, prepárate para:

- Internet se cae
- Base de datos no responde
- Usuario escribe mal un dato
- Se llena el disco duro
- Hackean el sistema

Solución:

- Mensajes de error claros
- RespalDOS automáticos
- Validaciones de datos
- Logs (registro de qué pasa)

H) PIENSA EN LA SEGURIDAD

- **Protege la información**

Analogía: Un banco tiene:

- Cámaras de seguridad
- Guardias
- Bóvedas con clave
- Alarmas

En tu sistema:

1. Autenticación

- Solo usuarios registrados entran

2. Autorización

- Cada usuario solo ve lo que debe ver
- Vendedor no puede ver reportes de gerencia

3. Encriptación

- Contraseñas cifradas
- Datos sensibles protegidos

4. Validaciones

- No confíes en lo que escriben los usuarios
- Verifica TODO antes de guardarlo

I) HAZ PRUEBAS CONSTANTES (TESTEABILIDAD)

- **Verifica que todo funcione**

Analogía: Antes de vender un carro, la fábrica prueba:

- ¿Enciende el motor?
- ¿Frenan bien?
- ¿Funcionan las luces?
- ¿Soporta un choque?

En tu sistema:

- Prueba cada módulo por separado
- Prueba todo junto
- Simula errores a propósito
- Pide a usuarios que lo prueben

J) DOCUMENTA TODO

- **Deja instrucciones claras**

Analogía: Un manual de usuario te explica:

- Cómo encender el aparato
- Qué hace cada botón
- Qué hacer si falla

En tu sistema documenta:

1. Para usuarios:

- Cómo usar el sistema
- Preguntas frecuentes

2. Para desarrolladores:

- Qué hace cada módulo
- Por qué tomaste decisiones

- Cómo instalar y configurar

3. Para administradores:

- Cómo hacer respaldos
- Cómo resolver problemas comunes

6. CHECKLIST: ¿TIENES UNA BUENA ARQUITECTURA?

- **Marca lo que cumples:**

- ☐ ¿Está dividido en módulos claros?
- ☐ ¿Cada módulo hace una sola cosa?
- ☐ ¿Puedo cambiar una parte sin romper todo?
- ☐ ¿Puede crecer si aumentan los usuarios?
- ☐ ¿Está documentado?
- ☐ ¿Es fácil de probar?
- ☐ ¿Está protegido contra errores?
- ☐ ¿Maneja la seguridad correctamente?
- ☐ ¿Los nombres son claros?
- ☐ ¿No hay datos duplicados en la BD?
- ☐ ¿Puede trabajar otro programador en esto sin preguntarte todo?

Entre más "SÍ" tengas, mejor es la arquitectura.

7. ELEMENTOS DE DISEÑO DE UNA ARQUITECTURA DE SOFTWARE

Piensa en la arquitectura de software como si fuera **construir una casa**. No puedes empezar a pegar ladrillos sin antes tener un plano, ¿cierto? La arquitectura de software es ese "plano" que define cómo se va a organizar todo tu sistema.

- **COMPONENTES**

¿Qué son? Son las piezas o módulos principales que conforman tu sistema.

Analogía: Son como las habitaciones de una casa (cocina, baño, sala, dormitorio). Cada uno tiene una función específica.

Ejemplo en Trueque Match:

- Componente de **Usuarios** (maneja registro, login)
- Componente de **Publicaciones** (crea y muestra ofertas)
- Componente de **Chat** (mensajería entre usuarios)
- Componente de **Trueques** (gestiona intercambios)

¿Por qué importan? Porque dividen el trabajo en partes manejables. Es más fácil arreglar la cocina que toda la casa a la vez.

• **CONECTORES**

¿Qué son? Son los "puentes" o canales de comunicación entre los componentes.

Analogía: Son como las puertas y pasillos que conectan las habitaciones de tu casa. Permiten que la gente (datos) se mueva de un lugar a otro.

Ejemplo en Trueque Match:

- **API REST:** El celular le pide datos al servidor
- **SQL Queries:** Node.js le pide datos a PostgreSQL
- **WebSockets:** Para el chat en tiempo real

¿Por qué importan? Sin conectores, los componentes estarían aislados. Es como tener habitaciones sin puertas: no sirven de nada.

• **INTERFACES**

¿Qué son? Son los "contratos" o reglas que definen cómo se comunican los componentes.

Analogía: Son como los enchufes de tu casa. Un enchufe tipo A solo acepta clavijas tipo A. No puedes meter cualquier cosa.

Ejemplo en Trueque Match:

- El frontend (app móvil) solo puede pedirle datos al backend usando endpoints específicos:

- /api/usuarios/login
- /api/publicaciones/crear
- /api/trueques/aceptar

¿Por qué importan? Porque garantizan que todos "hablen el mismo idioma". Si cambias algo interno, no rompes todo el sistema.

- **CONFIGURACIÓN (O TOPOLOGÍA)**

¿Qué son? Es la forma en que se organizan y distribuyen los componentes.

Define cómo se comunica todo y dónde vive cada cosa. Es la "vista aérea" de tu sistema.

Analogía: Es el plano de tu casa. Define dónde va cada habitación, cuántos pisos tiene, si hay sótano, etc.

Tipos comunes:

a) Cliente-Servidor (como Trueque Match)

- Celular (Cliente) → Router → PC Servidor → Base de datos

b) Capas (Layers)

```
Capa de Presentación (App)
    ↓
Capa de Negocio (Node.js)
    ↓
Capa de Datos (PostgreSQL)
```

c) Microservicios (para cuando creces)

Servicio de Usuarios + Servicio de Chat + Servicio de Pagos (cada uno independiente)

- **PATRONES ARQUITECTÓNICOS**

¿Qué son? Son soluciones probadas para problemas comunes en diseño de software.

Analogía: Son como los "estilos de casas" (moderna, colonial, minimalista). Cada estilo tiene sus reglas y características.

Ejemplos populares:

a) MVC (Modelo-Vista-Controlador)

- **Modelo:** Los datos (base de datos)
- **Vista:** Lo que ve el usuario (app móvil)
- **Controlador:** La lógica (backend que procesa)

b) REST API (el que usas en Trueque Match)

- Todo se comunica por HTTP
- Cada recurso tiene su URL
- Usa verbos estándar (GET, POST, PUT, DELETE)

¿Por qué importan? Porque no tienes que inventar la rueda. Son recetas probadas que ya funcionan.

• ESTILOS ARQUITECTÓNICOS

¿Qué son? Es el enfoque general que define cómo se comporta todo el sistema.

Analogía: Es decidir si tu casa será de un piso o dos pisos, si tendrá patio o no, si será abierta o con muchas divisiones.

Ejemplos:

a) Monolítica (todo junto, como Trueque Match ahora)

- Una sola aplicación que hace todo
- Fácil de empezar, pero crece complicado

b) Distribuida (para después, cuando escales)

- Partes del sistema en diferentes servidores
- Más complejo, pero más robusto

¿Por qué importa? Define cómo crecerá tu sistema en el futuro.

7. RESTRICCIONES Y REQUISITOS 🏛️

¿Qué son? Son las reglas y límites que debe cumplir tu sistema.

Analogía: Son como las leyes de construcción. No puedes hacer una casa de 10 pisos en un terreno que solo soporta 2.

Ejemplos en Trueque Match:

- **Rendimiento:** Responder en menos de 3 segundos
- **Seguridad:** Cifrar contraseñas
- **Disponibilidad:** Estar activo 99% del tiempo
- **Costo:** Usar solo software gratuito

¿Por qué importan? Porque definen qué SÍ puedes hacer y qué NO. Son los límites de tu proyecto.

8. VISTAS ARQUITECTÓNICAS

¿Qué son? Son diferentes perspectivas del mismo sistema.

Analogía: Es como los planos de una casa:

- **Vista desde arriba** (plano del arquitecto)
- **Vista de frente** (fachada)
- **Vista de instalaciones** (tuberías, cables)
- **Vista de decoración** (cómo se ve por dentro)

Ejemplos:

- **Vista Lógica** → Cómo se organizan las funcionalidades
- **Vista de Procesos** → Cómo fluyen los datos
- **Vista de Desarrollo** → Cómo se estructura el código
- **Vista Física** → Dónde está cada cosa (tu diagrama de despliegue)

¿Por qué importan? Porque diferentes personas necesitan ver diferentes cosas. El programador necesita ver el código, el cliente necesita ver cómo funciona, y el instructor necesita ver la estructura completa.

RESUMEN ELEMENTOS DE DISEÑO DE UNA ARQUITECTURA DE SOFTWARE

Elemento	¿Qué es?	Analogía
Componentes	Partes funcionales del sistema	Habitaciones de una casa
Conectores	Mecanismos de comunicación	Puertas y pasillos
Interfaces	Reglas para conectar componentes	Enchufes estándar
Configuración	Organización y parámetros del sistema	Plano de la casa
Patrones	Soluciones probadas y reutilizables	Estilos arquitectónicos (moderno, clásico)
Estilos	Enfoque general del sistema	Casa de uno o dos pisos
Restricciones	Límites técnicos o normativos	Leyes de construcción
Vistas	Perspectivas del sistema	Planos desde distintos ángulos

8. TIPOS DE ARQUITECTURA DE SOFTWARE

Las arquitecturas se clasifican según **cómo organizan los componentes del sistema, cómo se comunican y cómo se despliega el software.**

Aquí van los tipos principales:

- **Arquitectura en Capas (Layered Architecture)**

Es el tipo más usado y el que enseñan primero.

Divide el sistema en capas donde cada capa cumple una función específica y solo habla con la capa que tiene arriba o abajo.

Capas típicas:

- Presentación (UI)
- Lógica de negocio
- Acceso a datos
- Base de datos

Analogía:

Un pastel de 4 capas.

Cada capa tiene su sabor y propósito, pero juntas forman el producto final.

Ejemplo real:

Una app web con:

- Frontend (HTML, CSS, JS)
- Backend (Java, C#, Python)
- BD (MySQL)

- **Arquitectura Cliente – Servidor**

Modelo clásico de la web.

Es cuando el cliente (app, navegador, móvil) hace solicitudes y el servidor responde.

Analogía:

Un restaurante:

- Cliente pide el plato
- Cocina (servidor) lo prepara y lo entrega

Ejemplo real:

WhatsApp Web, Gmail, bancos, Netflix.

- **Arquitectura MVC (Modelo – Vista – Controlador)**

Muy usada en web y móviles.

Divide la app en tres partes:

- **Modelo:** maneja los datos
- **Vista:** interfaz con el usuario
- **Controlador:** la lógica que conecta ambas

Analogía:

Una tienda:

- Vista → vitrina
- Modelo → inventario
- Controlador → vendedor que coordina

Ejemplos:

Laravel, Django, Spring MVC, Ruby on Rails.

- **Arquitectura Basada en Servicios (SOA)**

(La mamá de las API y microservicios)

El sistema se divide en **servicios independientes** que se comunican entre sí.

Ejemplo:

Servicio de pagos, servicio de usuarios, servicio de notificaciones.

Analogía:

Un centro comercial donde cada local es un servicio independiente.

- **Arquitectura de Microservicios**

La evolución moderna de SOA.

Hace que cada funcionalidad grande se divide en microservicios pequeños e independientes.

Características:

- Se despliegan por separado
- Se escalan por separado
- Son súper flexibles

Analogía:

En vez de un bus gigante, tienes muchos taxis independientes.

Ejemplos:

Netflix, Amazon, Uber.

- **Arquitectura Orientada a Eventos (Event-driven)**

Usa eventos para comunicar componentes.

En vez de llamar directamente a otro módulo, el sistema publica un evento y otros módulos lo escuchan.

Analogía:

Una emisora:

- La radio transmite
- Los oyentes escuchan si quieren

Ejemplo:

Notificaciones, sensores IoT, sistemas financieros.

- **7. Arquitectura de Tuberías y Filtros (Pipes and Filters)**

Procesamiento en cadena.

La información pasa por una serie de filtros (etapas) donde se va transformando.

Analogía:

Una fábrica de agua filtrada:

Filtro 1 → Filtro 2 → Filtro 3.

Ejemplo:

Procesamiento de videos, audio, imágenes.

- **8. Arquitectura Monolítica**

Todo en un solo bloque.

Toda la lógica del sistema está en una única aplicación.

Ventajas:

Fácil de desarrollar al inicio.

Desventajas:

Difícil de escalar.

Ejemplo:

Apps tradicionales y sistemas viejos.

- **9. Arquitectura Microkernel (Plug-in Architecture)**

Sistema base + complementos.

Aquí hay un núcleo principal y módulos que se conectan como plugins.

Analogía:

Un celular con apps:

- El sistema operativo es el núcleo
- Las apps son los plugins

Ejemplos:

IDE como Eclipse, videojuegos, sistemas operativos.

- **10. Arquitectura Basada en Componentes**

Se construye con componentes reutilizables.

¿Qué es?

Cada módulo del sistema es un componente independiente, como piezas de Lego.

Ejemplo:

Componentes UI en React, Angular, Flutter.

- **11. Arquitectura N-Capas / N-Tier**

Muy usada en empresas grandes.

Es como la arquitectura en capas, pero cada capa puede estar en un servidor diferente.

Ejemplo:

- Servidor para la app
- Servidor para la BD
- Servidor para la API

Tabla general de Comparación tipos de arquitectura de software

Tipo de arquitectura	Descripción	Ventajas	Desventajas	Ejemplos reales
Monolítica	Toda la aplicación en un solo bloque.	Fácil de iniciar, sencilla de desplegar.	Difícil de escalar, difícil de mantener cuando crece.	Sistemas antiguos, apps pequeñas.
En Capas (Layered)	Divide la app en capas como: presentación, negocio, datos.	Orden, claridad, fácil de mantener.	Puede volverse rígida, más lenta.	Apps empresariales, bancos.
N-Capas / N-Tier	Capas distribuidas en servidores diferentes.	Más seguridad y rendimiento.	Más costosa y compleja.	Sistemas corporativos grandes.
Cliente – Servidor	Cliente envía peticiones y servidor responde.	Centraliza lógica y datos.	Dependencia del servidor.	Gmail, WhatsApp Web.
MVC	Divide en Modelo, Vista y Controlador.	Mejor orden visual y lógica.	Puede complicarse si crece.	Laravel, Django, Spring MVC.
Basada en Componentes	Reutiliza componentes como piezas tipo Lego.	Reutilización, modularidad.	Puede haber demasiados componentes.	React, Angular, Flutter.
SOA (Orientada a Servicios)	Servicios independientes que se comunican.	Integración, escalabilidad.	Configuración más compleja.	Empresas grandes, ERP.
Microservicios	Funcionalidades divididas en microservicios independientes.	Escalabilidad brutal, despliegues independientes.	Más compleja, requiere DevOps.	Netflix, Uber, Amazon.
Orientada a Eventos	Módulos que reaccionan a eventos.	Excelente para sistemas distribuidos.	Más difícil de depurar.	IoT, notificaciones, stocks.
Pipes and Filters	Procesa datos en una cadena de filtros.	Buen procesamiento de datos.	No sirve para todos los sistemas.	Edición de video/audio.
Microkernel (Plug-in)	Núcleo base + plugins o módulos externos.	Flexible, ampliable.	Núcleo debe ser muy estable.	Sistemas operativos, IDE.

9. ESCALABILIDAD EN ARQUITECTURA DE SOFTWARE (EXPLICACIÓN GENERAL Y RESUMIDA)

La **escalabilidad** es la capacidad que tiene un sistema de **crecer, manejar más usuarios, más datos o más procesos**, sin volverse lento ni dañarse.

La arquitectura del software define **cómo** escalar.

- **Existen dos formas principales de escalabilidad:**

1. Escalabilidad Vertical (Scale Up)

También llamada **escalar hacia arriba**.

¿En qué consiste?

En aumentar la potencia del mismo servidor.

Ejemplos:

- Más RAM
- Mejor procesador
- Más almacenamiento
- Hardware más rápido

Idea general:

“Fortalecer una sola máquina”.

Ventajas:

- Fácil de implementar
- No requiere cambiar la arquitectura

Desventajas:

- Tiene límite físico
- Se vuelve costosa rápidamente

2. Escalabilidad Horizontal (Scale Out)

También llamada **escalar hacia los lados**.

¿En qué consiste?

En **agregar más servidores o más instancias** que trabajen en paralelo.

Ejemplos:

- Pasar de 1 servidor a 3, a 10 o a cientos
- Añadir contenedores en Docker
- Balanceadores de carga

Idea general:

“Agregar más máquinas para repartir el trabajo”.

Ventajas:

- Prácticamente sin límite
- Ideal para sistemas grandes y modernos
- Más estable y tolerante a fallos

Desventajas:

- Requiere una arquitectura bien diseñada
- Es más compleja de administrar

Ejemplo:

- **Vertical:** Mejoro una moto (más motor, más potencia).
- **Horizontal:** En vez de una moto pongo 5 motos, luego 10 motos, y reparten el trabajo.